

map 정리

key와 value를 묶어 저장하고 key 기준으로 자동 정렬하는 컨테이너입니다.

사용처 (Where)

- 이름-점수, 번호-정보처럼 key로 값을 찾을 때
- key를 정렬된 순서로 관리할 때
- 빈도수 세기

방법 (How to Use)

- map<Key, Value>로 선언
- m[key]로 값 접근/삽입
- find로 존재 확인
- 순회하면 key 오름차순

기본 정보

- 필요 헤더: #include <map>
- 기본 선언: map<string, int> score;

왜 써야 할까? (Why)

- key로 데이터를 찾는 코드가 매우 단순해짐
- 정렬된 key 순서가 필요할 때 좋음
- 빈도수 계산을 쉽게 작성할 수 있음

주요 함수

함수/문법	의미
m[key]	key의 value 접근
insert({k,v})	쌍 삽입
find(k)	key 검색
erase(k)	key 삭제
count(k)	존재 여부

기본 코드 예시

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<string, int> score;
    score["Kim"] = 90;
    score["Lee"] = 80;

    cout << score["Kim"] << '\n';
}
```

map 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
삽입/검색/삭제	$O(\log n)$
순회	$O(n)$
<code>m[key]</code>	없으면 기본값 생성

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	이미 구현된 기능 사용	자료구조 설계와 예외 처리 필요	배열/포인터 직접 관리
코드 길이	짧고 읽기 쉬움	길고 버그 가능성 증가	간단하지만 확장성이 낮음
성능	일반 상황에서 충분히 최적화	구현 실력에 따라 달라짐	고정 크기에서는 단순함
안정성	범위/메모리 관리 부담 감소	메모리/인덱스 실수 가능	범위 초과 위험 큼
추천 상황	대부분의 C++ 코드	자료구조 원리 학습	아주 단순한 고정 크기 데이터

응용: 단어 빈도수 세기

문자열이 나올 때마다 `map[word]`를 1 증가시키면 빈도표가 만들어집니다.

```
#include <iostream>
#include <map>
#include <vector>
using namespace std;

int main() {
    vector<string> words = {"cpp", "stl", "cpp", "map"};
    map<string, int> cnt;

    for (string w : words) cnt[w]++;

    for (auto item : cnt) {
        cout << item.first << ": " << item.second << '\n';
    }
}
```

처음 배울 때 자주 하는 실수

- `m[key]`는 key가 없으면 새로 만듦. 확인만 할 때는 `find` 사용
- 정렬이 필요 없고 속도가 중요하면 `unordered_map` 검토
- key는 중복될 수 없음

비슷한 항목과 차이

- unordered_map은 평균 $O(1)$ 이지만 정렬되지 않음
- set은 key만 저장
- multimap은 같은 key를 여러 개 저장 가능

한 줄 정리: map은(는) '이름-점수, 번호-정보처럼 key로 값을 찾을 때' 상황에서 먼저 떠올리면 좋은 STL입니다.